DS	**Session**	Tree 1	**Question Information**	Level 1 Challenge 69

Question description

A forest is an undirected graph without cycles (not necessarily connected).

Mohana and john are friends in kerala, both of them have a forest with nodes numbered from 1 to n, and they would like to add edges to their forests such that:

- After adding edges, both of their graphs are still forests.
- They add the same edges. That is, if an edge (u,v) is added to Mohana's forest, then an edge (u,v) is added to john's forest, and vice versa.

Mohana and johns want to know the maximum number of edges they can add, and which edges to add.

Constraints:

$1 \le n \le 105$,

$0 \le m1$

$m2 < n$

$1 \le u,v \le n$, $u \ne v$

Input

The first line contains three integers n, $m1$ and $m2$ — the number of nodes and the number of initial edges in Mohana's forest and john's forest.

Each of the next $m1$ lines contains two integers u and v — the edges in Mohana's forest.

Each of the next $m2$ lines contains two integers u and v ($1 \le u,v \le n$, $u \ne v$) — the edges in Johns's forest.

Output

The first line contains only one integer h, the maximum number of edges Mohana and John can add.

Each of the next h lines contains two integers u and v ($1 \le u, v \le n$, $u \ne v$) — the edge you add each time.

If there are multiple correct answers, you can print any one of them.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 3 2
5 4
2 1
4 3
4 3
1 4
```

EXPECTED OUTPUT

```
1
1 5
```

Test Case 2

INPUT (STDIN)

```
5 3 2
5 3
2 3
2 3
2 3
2 4
```

EXPECTED OUTPUT

```
1
1 2
4 5
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(m1--)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

500

Test Case 3

NLOC

30

Code

solution.c

```c
#include <stdio.h>

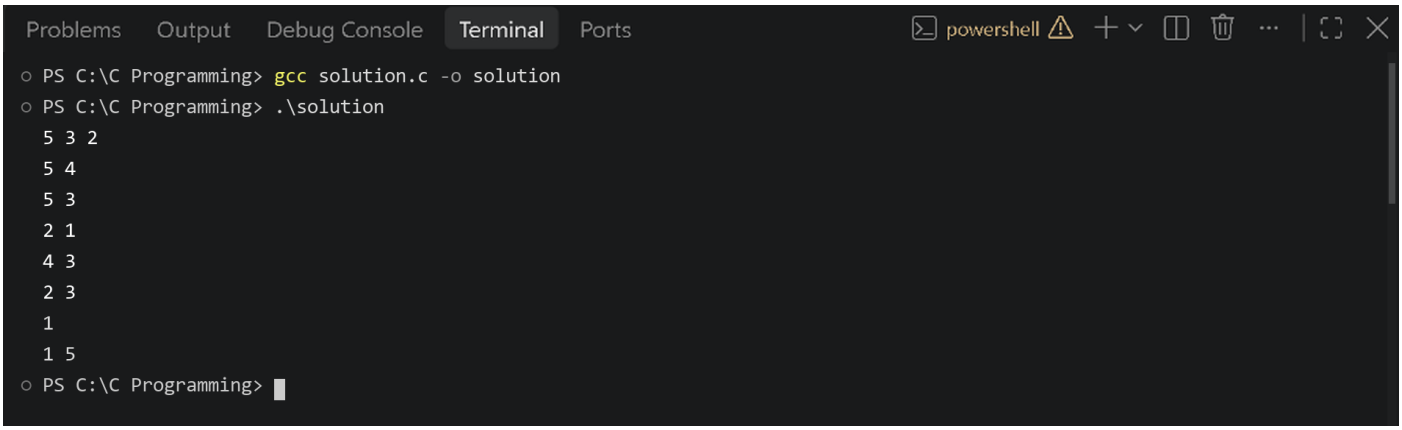
struct DSU {
    int parent[1005];
} dsu1, dsu2;

int find(struct DSU *d, int x) {
    if (d->parent[x] == x) return x;
    return d->parent[x] = find(d, d->parent[x]);
}

int unite(struct DSU *d, int a, int b) {
    int rootA = find(d, a), rootB = find(d, b);
    if (rootA == rootB) return 0;
    d->parent[rootA] = rootB;
    return 1;
}

int main() {
    int n, m1, m2;
    if (scanf("%d %d %d", &n, &m1, &m2) != 3) return 0;
    for (int i = 1; i <= n; i++) {
        dsu1.parent[i] = i;
        dsu2.parent[i] = i;
    }
    while (m1--) {
        int u, v; scanf("%d %d", &u, &v);
        unite(&dsu1, u, v);
    }
    while (m2--) {
        int u, v; scanf("%d %d", &u, &v);
        unite(&dsu2, u, v);
    }
    int addU[1005], addV[1005], ans = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = i + 1; j <= n; j++) {
            if (find(&dsu1, i) != find(&dsu1, j) && find(&dsu2, i) != find(&dsu2, j)) {
                unite(&dsu1, i, j);
                unite(&dsu2, i, j);
                addU[ans] = i;
                addV[ans] = j;
                ans++;
            }
        }
    }
    printf("%d\n", ans);
    for (int i = 0; i < ans; i++) printf("%d %d\n", addU[i], addV[i]);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5 3 2
5 4
5 3
2 1
4 3
2 3
1
1 5
PS C:\C Programming>